

CMU-CS 462: Software Measurement and Analysis

Spring 2025-2026

Nguyen Duc Man

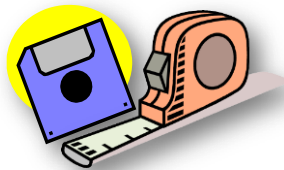
E: mannd@duytan.edu.vn

T: 0904235945

12/01/2026

1

Measuring Internal Product Attributes: Software Size



12/01/2026

2

2

Contents

1. Software size
2. Size: Length (code, specification, design)
3. Size: Reuse
4. Size: Functionality (function point, feature point, object point, use-case point)
5. Size: Complexity



12/01/2026

3

3

What is software size?

- ▶ Size (surface) of a wall: **square feet**;
- ▶ Size (volume) of a bottle: **liters**;
- ▶ Size (weight) of a man: **pounds**.

International
Standards

- ▶ **Software size ??**

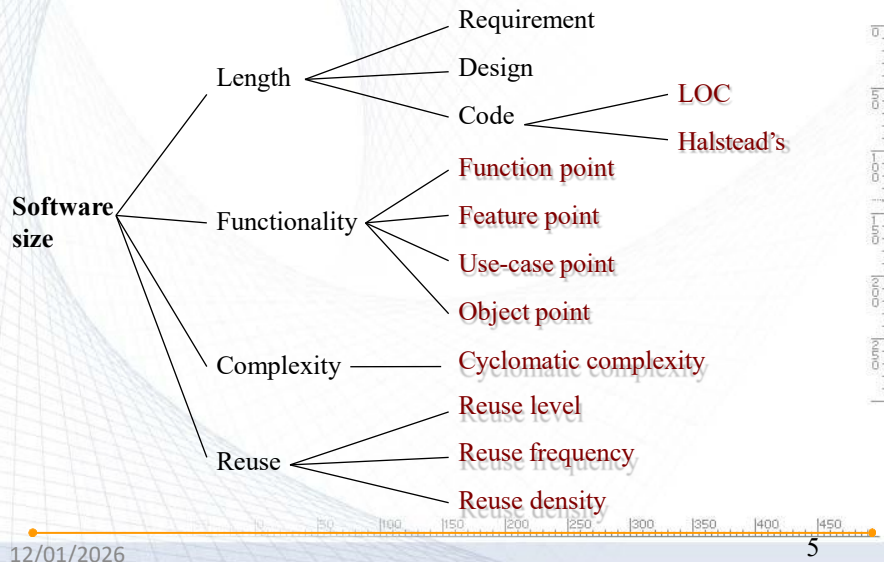
- ▶ What is the square feet metric in software?
- ▶ We need **standards** to express the size of software.

12/01/2026

4

4

Software Size Metrics: Summary



5

Software Size /1

- Internal product attributes describe a software product in a way that is dependent only on the product itself.
- One of the most useful attributes is the **size** of a software product, which can be measured statically, i.e., without executing the system.
- It is necessary to define software size in terms of more than one internal attributes, each capturing a key aspect of software size.
- Size measurement must reflect *effort*, *cost* and *productivity*.

6

Software Size /2

- Software size can be described by *length*, *functionality*, and *complexity*.
 - *Length* is the physical product size.
 - *Functionality* is a measure of the functions supplied by the product to the user.
 - *Complexity* is a multi-faceted attribute which can be interpreted in multiple ways.
- *Reuse* is also an issue in size, specifically the amount or size of reuse within a program.

12/01/2026

7

7

Software Size: Length

- Length is the “physical size” of the product.
- In a software development effort, there are three major development products: **specification**, **design**, and **code**.
- The length of the specification can indicate how long the design is likely to be, which in turn is a predictor of code length.
- Traditionally, code length refers to text-based code length.

12/01/2026

8

8

Length: Code – LOC /1

- The most commonly used measure of source code program length is the number of lines of code (LOC).
 - *NCLOC*: non-commented source line of code or effective lines of code (ELOC).
 - *CLOC*: commented source line of code.
- By measuring NCLOC and CLOC separately we can define:

$$\text{total length (LOC)} = \text{NCLOC} + \text{CLOC}$$
- The ratio: CLOC/LOC measures the density of comments in a program.

12/01/2026

9

9

Length: Code – LOC /1

Lines of Code (LOC): As the name suggest, LOC count the total number of lines of source code in a project.

The units of LOC are:

- KLOC- Thousand lines of code
- NLOC- Non comment lines of code
- KDSI- Thousands of delivered source instruction

12/01/2026

10

10

Exercise

Consider this snippet of C code as an example of the ambiguity encountered when determining SLOC:

```
for (i = 0; i < 100; i += 1) printf("hello"); /* How many lines of code is this? */
```

In this example we have:

- 1 Physical Lines of Code (LOC)
- 2 Logical Line of Code (LLOC) (for statement and printf statement)
- 1 comment line

12/01/2026

11

11

Exercise

Depending on the programmer and/or coding standards, the above "line of code" could be written on many separate lines:

```
for (i = 0; i < 100; i += 1)
{
    printf("hello");
} /* Now how many lines of code is this? */
```

In this example we have:

- 4 Physical Lines of Code (LOC): is placing braces work to be estimated?
- 2 Logical Line of Code (LLOC): what about all the work writing non-statement lines?
- 1 comment line: tools must account for all code and comments regardless of comment placement.

12/01/2026

12

12

Exercise

```
for (i = 0; i < 100; i += 1)
{
    printf("hello");

} /* Now how many lines of code is this? */
```

In this example we have:

- 5 Physical Lines of Code (LOC): is placing braces work to be estimated?
- 2 Logical Line of Code (LLOC): what about all the work writing non-statement lines?
- 1 comment line: tools must account for all code and comments regardless of comment placement.

12/01/2026

13

13

LLOC

Logical Lines Of Code is the number of programming language statements (also called Effective Lines Of Code, eLoc, eSLOC, SLOC-L) in the code. What comprises a code statement is language dependent, for C language "i = 5;" is a single statement.

The LLOC count variant used by ProjectCodeMeter includes the following improvements:

- **auto-generated code lines are ignored**
- **header files are ignored**
- **ineffective code statements are ignored**
- **pragma compiler directives are ignored**
- **labels are ignored**
- **switch cases are statements by themselves, not part of the next LLOC**
- **several statements on the same physical line are counted as several LLOC**

12/01/2026

14

14



Logical Lines Of Code is the number of programming language statements (also called Effective Lines Of Code, eLoc, eSLOC, SLOC-L) in the code. What comprises a code statement is language dependent, for C language "i = 5;" is a single statement.

The LLOC count variant used by ProjectCodeMeter includes the following improvements:

- **auto-generated code lines are ignored**
- **header files are ignored**
- **ineffective code statements are ignored**
- **pragma compiler directives are ignored**
- **labels are ignored**
- **switch cases are statements by themselves, not part of the next LLOC**
- **several statements on the same physical line are counted as several LLOC**

12/01/2026

15

15



- **auto-generated code lines are ignored**
- **header files are ignored**
- **ineffective code statements are ignored**
- **pragma compiler directives are ignored**
- **labels are ignored**
- **switch cases are statements by themselves, not part of the next LLOC**
- **several statements on the same physical line are counted as several LLOC**

Example:

```
for (i=0; i < 5; i++){ //this line counts as 3 LLOCs by ProjectCodeMeter
    CreateUser(i); //the line counts as 1 LLOC
} //this line counts as 0 LLOC
```

12/01/2026

16

16

Length: Code – LOC /2

Variations of LOC:

- Count of physical lines including blank lines.
- Count of all lines except blank lines and comments.
- Count of all statements except comments (statements taking more than one line count as only one line).
- Count of all lines except blank lines, comments, declarations and headings.
- Count of only executable statements, not including exception conditions.

12/01/2026

17

17

Length: Code – LOC /3

■ Measurement Unit: Lines of Source Code

Statement Type	Includes	Excludes
Executable	X	
Non-executable		
Declarations	X	
Compiler Directives	X	
Comments		X
On their own lines		X
On lines with source code		X
Banners and nonblank spacers		X
Blank (empty) comments		X
Blank Lines		X

12/01/2026

18

18

Length: Code – LOC /4

■ Advantages of LOC

- Simple and automatically measurable
- Correlates with programming effort (& cost)

■ Disadvantage of LOC

- Vague definition
- Language dependability
- Not available for early planning
- Developers' skill dependability
- Encouraging “sumo” development!



12/01/2026

19

19

Length: Halstead's Work

/1

■ Maurice Halstead's Theory (1971~1979):



- A program P is a collection of tokens, composed of two basic elements: **operands** and **operators**
- **Operands** are variables, constants, addresses
- **Operators** are defined operations in a programming language (language constructs)

if ... else
+ - > = ;
main()
goto

a, b, x
100

12/01/2026

20

20

Length: Halstead's Work

/2

- Number of distinct operators in the program (μ_1)
- Number of distinct operands in the program (μ_2)
- Total number of occurrences of operators in the program (N_1)
- Total number of occurrences of operands in the program (N_2)

- **Program vocabulary** (μ)

$$\mu = \mu_1 + \mu_2$$

12/01/2026

21

21

Length: Halstead's Work

/3

- **Program length** is the total number of occurrences of operators and operands:

$$N = N_1 + N_2$$

- **Program volume** is the number of mental comparisons needed to write a program of length N

$$V = N \log_2 \mu = (N_1 + N_2) \log_2 (\mu_1 + \mu_2)$$

- **Program level** (L):

$$L = V^*/V \quad \text{or} \quad L = \frac{1}{D} = \frac{2}{\mu_1} \times \frac{\mu_2}{N_2}$$

- where V^* is the minimum size potential volume (i.e., minimal size of implementation) and D is program difficulty

12/01/2026

22

22

- **Program length** is the total number of occurrences of operators and operands:

$$N = N_1 + N_2$$

- **Program estimated length** ($\hat{N}$)

$$\hat{N} = \mu_1 \log_2 \mu_1 + \mu_2 \log_2 \mu_2$$

- **Effort** required to generate program P : number of elementary discriminations

$$E = V \times D = \frac{V}{L} = \frac{\mu_1 N_2}{2\mu_2} \times N \log_2 \mu$$

12/01/2026

23

23

- **Time** required for developing program P is the total effort divided by the number of elementary discriminations per second

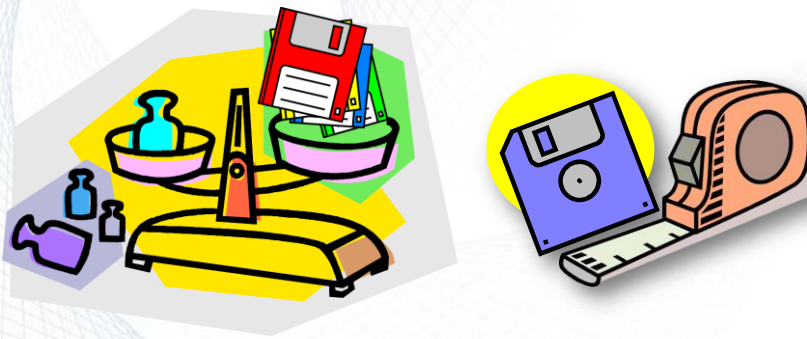
$$T = E / \beta$$

- In cognitive psychology β is usually a number between 5 and 20
- Halstead claims that $\beta = 18$

12/01/2026

24

24



Measuring Software Size!

12/01/2026

25

25

Length: Halstead's Work

/6

- **Remaining bugs:** the number of bugs left in the software at the delivery time

$$B = E^{2/3} / 3000$$

- **Conclusion:** the bigger program needs more time to be developed and more bugs remained

12/01/2026

26

26

Example 1

- For the following C program:

```
#include<stdio.h>
main()
{
  int a ;
  scanf ("%d", &a);
  if ( a >= 10 )
  if ( a < 20 ) printf ("10 < a < 20 %d\n" , a);
  else      printf ("a >= 20 %d\n" , a);
  else      printf ("a <= 10 %d\n" , a);
}
```

Operands

12/01/2026

27

27

Example 1 (cont'd)

- Determine number of operators (μ_1).
- Determine number of operands (μ_2).
- Determine the program length in terms of the total number of occurrences of operators (N_1) and operands (N_2): $N = N_1 + N_2$
- Estimate program length

12/01/2026

28

28

Example 1 (cont'd)

Operators	Number of occurrences	Operators	Number of occurrences
#	1	<=	1
include	1	\n	3
stdio.h	1	printf	3
< ... >	1	<	3
main	1	>=	2
(...)	7	if ... else	2
{ ... }	1	&	1
int	1	,	4
;	5	%d	4
scanf	1	" ... "	4
$\mu_1 = 20$		$N_1 = 47$	

12/01/2026

29

29

Example 1 (cont'd)

Operands	Number of occurrences
a	10
10	3
20	3
$\mu_2 = 3$	$N_2 = 16$
$\mu_1 = 20$	$N_1 = 47$
Program length: $N = N_1 + N_2 = 63$	
Program Estimated length: $\hat{N} = \mu_1 \log_2 \mu_1 + \mu_2 \log_2 \mu_2 = 20 \log_2 20 + 3 \log_2 3 = 91.1934$	

12/01/2026

30

30

Example 2

- For the following program calculate Halstead's (c1) number of operators; (c2) number of operands; (c3) program vocabulary; (c4) occurrences of operators in the program; (c5) occurrences of operands in the program; (c6) program length; (c7) program volume; (c8) program estimated length.

```

1: read x,y,z;
2: type = "scalene";
3: if (x == y or x == z or y == z) type ="isosceles";
4: if (x == y and x == z) type ="equilateral";
5: if (x >= y+z or y >= x+z or z >= x+y) type ="not a triangle";
6: if (x <= 0 or y <= 0 or z <= 0) type ="bad inputs";
7: print type;

```

12/01/2026

31

31

Example 2 (cont'd)

12/01/2026

32

32

Example 2 (cont'd)

- (c1) Number of distinct operators in the program: $\mu_1 = 14$
- (c2) Number of distinct operands in the program: $\mu_2 = 6$
- (c3) Program vocabulary: $\mu = \mu_1 + \mu_2 = 20$
- (c4) Total number of occurrences of operators in the program:
 $N_1 = 51$
- (c5) Total number of occurrences of operands in the program:
 $N_2 = 39$

12/01/2026

33

33

Example 2 (cont'd)

- (c6) Program length: $N = N_1 + N_2 = 90$
- (c7) Program volume: $V = N \log_2 \mu = 90 \log_2 (20) = 388.9735$
- (c8) Program estimated length:
 $\hat{N} = \mu_1 \log_2 \mu_1 + \mu_2 \log_2 \mu_2 = 14 \log_2 14 + 6 \log_2 6 = 68.81274$

12/01/2026

34

34

Critics of Halstead's work

- Developed in the context of assembly languages and too fine grained for modern programming languages.
- The treatment of basic and derived measures is somehow confusing.
- The notions of time to develop and remaining bugs are arguable.
- Unable to be extended to include the size for specification and design.

12/01/2026

35

35

•Calculate number of physical LOC, NCLOC, CLOC and Cyclomatic Complexity of the following program:

```
FindMean(float Mean, FILE ScoreFile)
{ SumOfScores = 0.0; NumberOfScores = 0; Mean = 0;
  Read(ScoreFile, Score);          /*Read in and sum the scores*/
  while (! EOF(ScoreFile) ) {
    if ( Score > 0.0 ) {
      SumOfScores = SumOfScores + Score;
      NumberOfScores++;
    }
    Read(ScoreFile, Score);
  }
  /* Compute the mean and print the result */
  if (NumberOfScores > 0 ) {
    Mean = SumOfScores/NumberOfScores;
    printf("The mean score is %f\n", Mean);
  } else
    printf("No scores found in file\n");
}
```

12/01/2026

36

36

•Calculate number of physical LOC, NCLOC, CLOC and Cyclomatic Complexity of the following program:

```
FindMean(float Mean, FILE ScoreFile)
{ SumOfScores = 0.0; NumberOfScores = 0; Mean = 0;
  Read(ScoreFile, Score);          /*Read in and sum the scores*/
  while (! EOF(ScoreFile) ) {
    if ( Score > 0.0 ) {
      SumOfScores = SumOfScores + Score;
      NumberOfScores++;
    }
    Read(ScoreFile, Score);
  }
  /* Compute the mean and print the result */
  if (NumberOfScores > 0 ) {
    Mean = SumOfScores/NumberOfScores;
    printf("The mean score is %f\n", Mean);
  } else
    printf("No scores found in file\n");
}
```

12/01/2026

37

37

```
while (foo = getNext(bar)) {
  filterThis = false;
  if (condition1) {
    doSomething();      // doing something
    filterThis = true
    if (condition2) {doSomethingElse();filterThis=glob();}
  }
  if (! filterThis) process(foo);
}
```

12/01/2026

38

38

Work in group 4 : Discuss and answer the following question.
(Làm việc nhóm 4 sinh viên, thảo luận và trả lời câu hỏi sau)

Is Lines of Code is a Bad Metric or a Good Metric? Why?(10 mins)

How many lines of code per day is good?

Do lines of code matter?

Is 1000 lines of code a lot?

How many LOC per function?

12/01/2026

39

39

Measuring Software Size: Function Point (FP), Feature Point, Object Point and Use- case Point

12/01/2026

40

40

Review: Software Size

- Size measurement must reflect *effort*, *cost* and *productivity*.
- Size-oriented metrics are direct measures of software (include effort (time), money spent, LOC, pages of documents created, errors, and number of staff.)
- Defining software size in terms of **length**, **functionality**, and **complexity**, each capturing a key aspect of software size.
- Basic measure for length is LOC. Simple size-oriented metrics can be generated from LOC, such as:
 - $\text{Productivity} = \text{KLOC} / \text{person-month}$
 - $\text{Quality} = \text{defects} / \text{KLOC}$
 - $\text{Documentation} = \text{pages of documents} / \text{KLOC}$
 - etc.



12/01/2026

41

41

Length: Problem with LOC

- Depending on the programmer and/or coding standards, the "line of code" could be, and usually is, written on many separate lines:

```
for (i=0; i<100; ++i)
{
    printf("hello");
} /* Now how many lines of code is this? */
```

- In this example we have:
 - 4 Physical Lines of Code (is placing braces worth to be estimated?)
 - 2 Logical Line of Code (What about all the work writing non-statement lines?)
 - 1 Comment Line (?)

12/01/2026

42

42

Function-Oriented Metrics

- **Function Point (FP) is a weighted measure of software functionality.**
- The idea is that a product with more functionality will be larger in size.
- Function-oriented metrics are indirect measures of software which focus on functionality and utility.
- The first function-oriented metrics was proposed by Albrecht (1979~1983) who suggested a productivity measurement approach called the **Function Point (FP)** method.
- Function points (FPs) measure the amount of functionality in a system based upon the *system specification*.

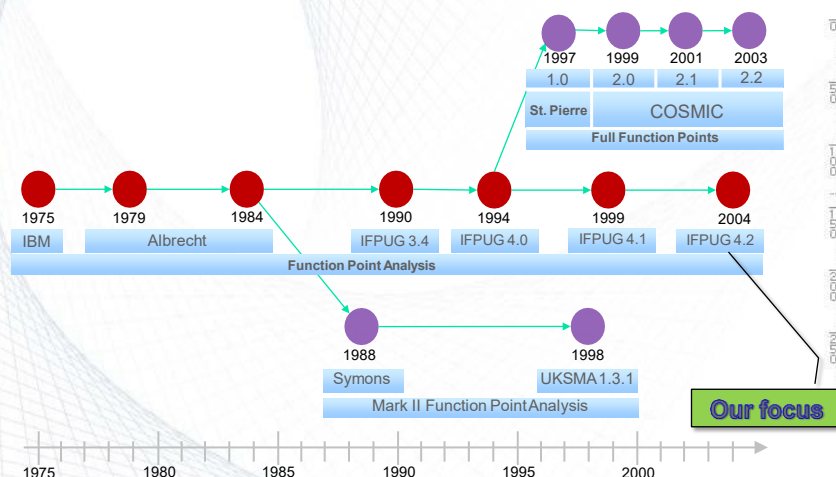
Estimation before implementation!

12/01/2026

43

43

FP: History



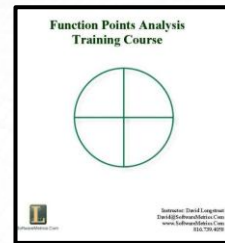
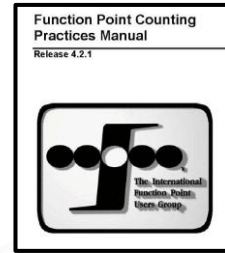
12/01/2026

44

44

FP: Current Status

- **1979:** Proposed by Albrecht, at IBM
- **1983:** Gained popularity, Albrecht's IEEE paper
- **1986:** International FP user group established (<http://www.ifpug.org>)
- **1990~2004:** IFPUG, Guidelines to FP measurement
- **1998:** ISO 14143-FSM: Functional Size Measurement
- **1999:** IFPUG, FP counting practice manual, version 4.1
- **2004:** IFPUG, FP counting practice manual, version 4.2



12/01/2026

45

45

Function Point (FP) Standards

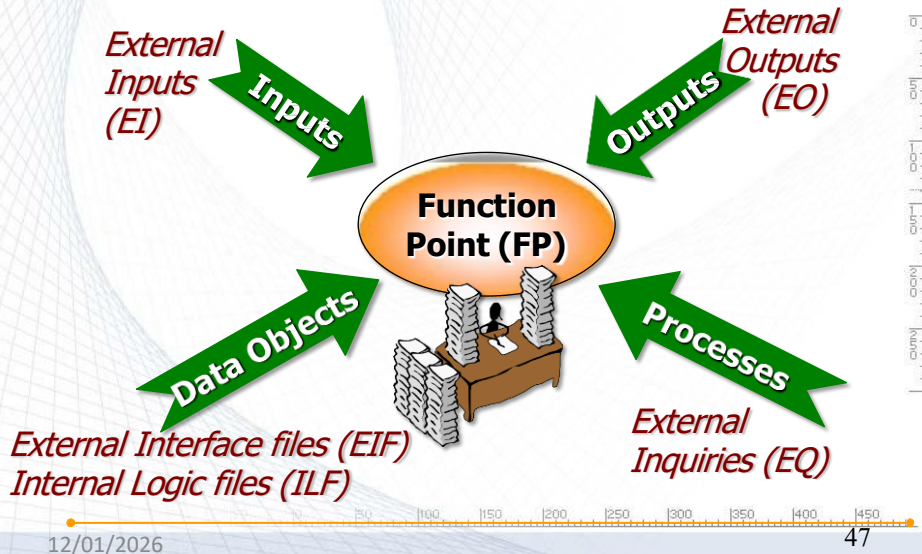
Standard	Name	Content
ISO/IEC 14143-1:2000	IFPUG 4.1	Unadjusted functional size measurement method -- Counting practices manual IFPUG Function Points VS 4.1 (unadjusted)
ISO/IEC 20926:2004	IFPUG 4.2	Unadjusted functional size measurement method -- Counting practices manual IFPUG Function Points VS 4.2 (unadjusted)
ISO/IEC 19761:2003	COSMIC-FFP	A functional size measurement method COSMIC Full Function Points Vs. 2.2
ISO/IEC 20968:2002	Mark II Function Point Analysis	Counting Practices Manual Mark II Function Points

12/01/2026

46

46

Function Point (FP) /1



47

Function Point (FP) /2

- Function Point (FP) is a weighted measure of software functionality.
- FP is computed in two steps:
 - 1) Calculating **Unadjusted Function point Count (UFC)**.
 - 2) Multiplying the UFC by a **Value Adjustment Factor (VAF)**.
- The final (adjusted) Function Point is:

$$FP = UFC \times VAF$$

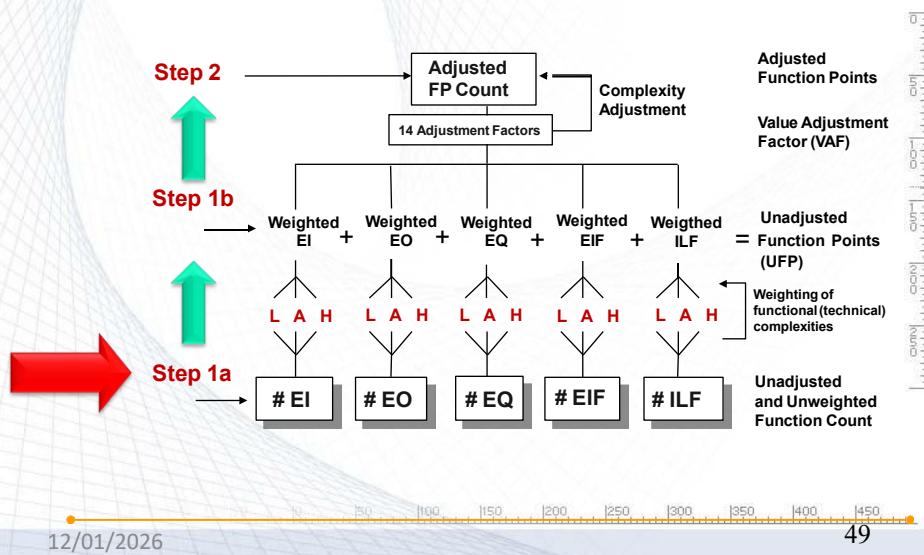
IFPUG Terminology

12/01/2026

48

48

FP Counting Summary

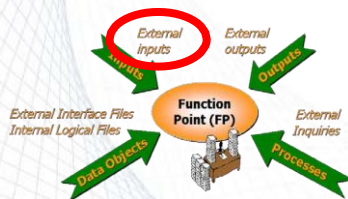


49

1. External Inputs(EI)

■ *External Inputs – IFPUG Definition:*

- An external input (EI) is an elementary process that processes data or control information that comes from outside the application boundary
- The primary intent of an EI is to maintain one or more ILFs and/or to alter the behavior of the system



■ **Example:**

- Data entry by users
- Data or file feeds by external applications

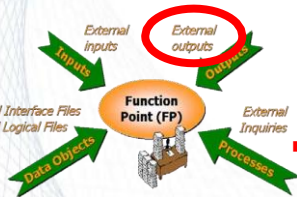
12/01/2026

50

50

2. External Outputs(EO)

■ *External Outputs – IFPUG Definition:*



- An external output (EO) is an elementary process that sends data or control information outside the application boundary
- The primary intent of an external output is to present information to a user through processing logic other than, or in addition to, the retrieval of data or control information

■ **Example:**

- Reports created by the application being counted, where the reports include derived information

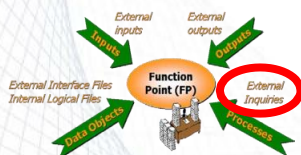
12/01/2026

51

51

3. External Inquiries(EQ)

■ *External Inquiries – IFPUG Definition:*



- An external inquiry (EQ) is an elementary process that sends data or control information outside the application boundary
- The primary intent of an external inquiry is to present information to a user through the retrieval of data or control information from an ILF or EIF
- The processing logic contains no mathematical formulas or calculations, and creates no derived data

■ **Example:**

- Reports created by the application being counted, where the report does not include any derived data

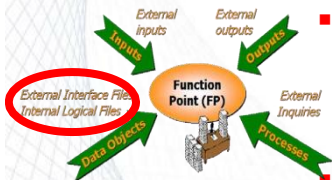
12/01/2026

52

52

4. Internal Logical Files (ILF)

■ *Internal Logical Files – IFPUG* *Definition:*



- An ILF is a user-identifiable group of logically related data or control information maintained within the boundary of the application
- The primary intent of an ILF is to hold data maintained through one or more elementary processes of the application being counted

■ **Example:**

- Tables in a relational database
- Files
- Application control information, perhaps things like user preferences that are stored by the application

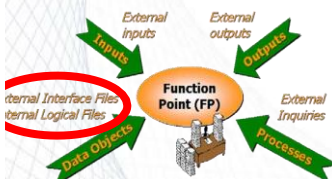
12/01/2026

53

53

5. Ext. Interface Files (EIF)

■ *External Interface files – IFPUG* *Definition:*



- An external interface file (EIF) is a user identifiable group of logically related data or control information referenced by the application, but maintained within the boundary of another application
- The primary intent of an EIF is to hold data referenced through one or more elementary processes within the boundary of the application counted

■ **Example:**

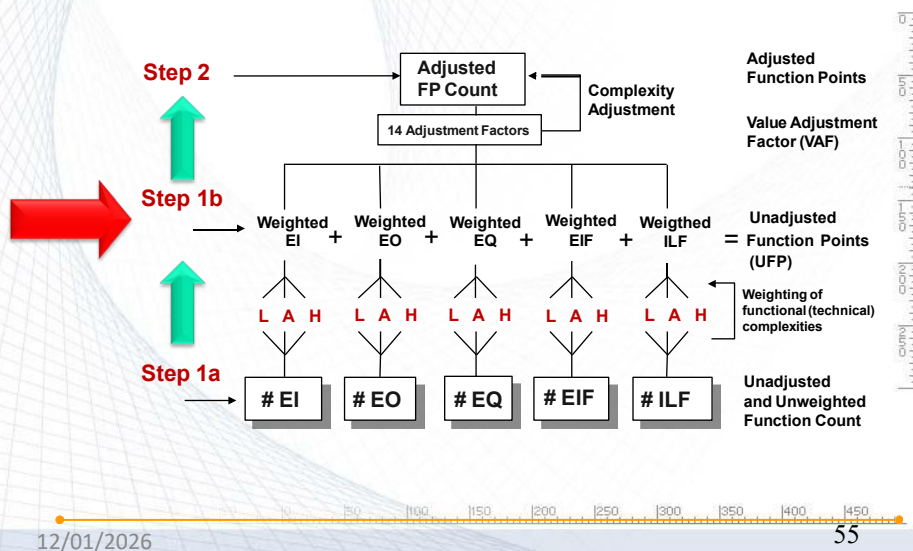
- As for ILF, but maintained in a different system

12/01/2026

54

54

FP Counting Summary



55

Unadjusted FP Count(UFC)

- A complexity rating is associated with each count according to *function point complexity weights*, below:

$$UFC = 4N_{EI} + 5N_{EO} + 4N_{EQ} + 7N_{EIF} + 10N_{ILF}$$

Element	Complexity Weighting Factor		
	Low (Simple)	Average	High (Complex)
External inputs (N_{EI})	3	4	6
External outputs (N_{EO})	4	5	7
External inquiries (N_{EQ})	3	4	6
External interface files (N_{EIF})	5	7	10
Internal logical files (N_{ILF})	7	10	15

More on weights in FP count manual

12/01/2026

56

56

FP Counting: Weighting of Technical Complexity

Elements	Complexity Weighting Factor			
	low	average	high	Sum
External Inputs (EI)	— x 3 = ____	— x 4 = ____	— x 6 = ____	
External Outputs (EO)	— x 4 = ____	— x 5 = ____	— x 7 = ____	
External Inquiries (EQ)	— x 3 = ____	— x 4 = ____	— x 6 = ____	
External Interface Files (EIF)	— x 5 = ____	— x 7 = ____	— x10 = ____	
Internal Logical Files (ILF)	— x 7 = ____	— x10 = ____	— x15 = ____	
Unadjusted Function Points (UFP)				

12/01/2026

57

57

Function Types –Complexity Assessment

- How to assess complexity?

	Data Function Types		Transaction Function Types		
	Internal Logical Files (ILF)	External Interface Files (EIF)	External Input (EI)	External Output (EO)	External Inquiry (EQ)
Elements evaluated for technical complexity assessment	REcord Types (RET): User recognizable sub groups of data elements within an ILF or an EIF. It is best to look at logical groupings of data to help identify them.		File Type Referenced (FTR): File type referenced by a transaction. An FTR must be an Internal Logical File (ILF) or External Interface File (EIF).		
	Data Element Types (DET): A unique user recognizable, non-recursive (non-repetitive) field containing dynamic information. If a DET is recursive then only the first occurrence of the DET is considered not every occurrence.				

12/01/2026

58

58

Complexity Assessment: EI

EI		#DET		
		1-4	5-15	> 15
#FTR	1	low (3)	low (3)	average (4)
	2	low (3)	average (4)	high (6)
	> 2	average (4)	high (6)	high (6)

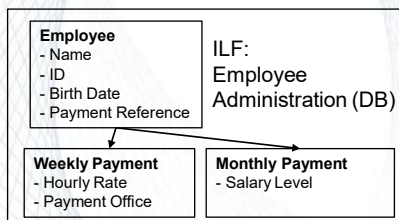
- Identify number of File Types Referenced (FTR)
- Identify number of Data Element Type (DET)
- Determine complexity weight (→ used to calculate FP count)

12/01/2026

59

59

External Input –Example



1 FTR
5 DET

EI		#DET		
		1-4	5-15	> 15
#FTR	1	low (3)	low (3)	average (4)
	2	low (3)	average (4)	high (6)
	> 2	average (4)	high (6)	high (6)

- Enter a new employee with Monthly Payment:

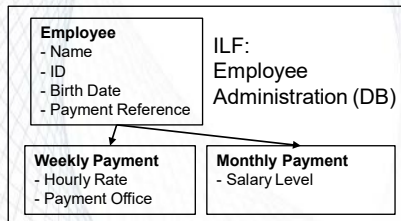
- Name
- ID
- Birth Date
- Payment Reference
- Salary Level

12/01/2026

60

60

EI – Example



1 FTR
6 DET

EI		#DET		
		1-4	5-15	> 15
#FTR	1	low (3)	low (3)	average (4)
	2	low (3)	average (4)	high (6)
	> 2	average (4)	high (6)	high (6)

12/01/2026

61

61

Complexity Assessment: EO

EO		#DET		
		1-5	6-19	> 19
#FTR	1	low (4)	low (4)	average (5)
	2-3	low (4)	average (5)	high (7)
	> 3	average (5)	high (7)	high (7)

- Identify number of File Types Referenced (FTR)
- Identify number of Data Element Type (DET)
- Determine complexity weight (→ used to calculate FP count)

12/01/2026

62

62

EO – Example

- Report of all Employees containing Names and Birth Dates, sorted by age.

ILF: Employee
- Name
- ID
- Birth Date
- Payment Reference

≤ 40 Years
Jerome Iginla, 10-02-1966
Rehet Warner, 05-03-1966
≤ 45 Years
Chris Chelios, 23-03-1961

1 FTR
2 DET

EO		#DET		
		1-5	6-19	> 19
#FTR	1	low (4)	low (4)	average (5)
	2-3	low (4)	average (5)	high (7)
	> 3	average (5)	high (7)	high (7)

12/01/2026

63

63

Complexity Assessment: EQ

EQ		#DET		
		1-5	6-19	> 19
#FTR	1	low (3)	low (3)	average (4)
	2-3	low (3)	average (4)	high (6)
	> 3	average (4)	high (6)	high (6)

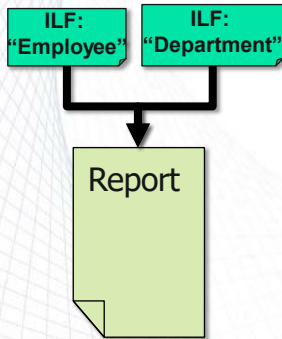
- Identify number of File Types Referenced (FTR)
- Identify number of Data Element Type (DET)
- Determine complexity weight (→ used to calculate FP count)

12/01/2026

64

64

EQ – Example



- Report of all employees belonging to Department X containing Names, Birth Dates, and showing the Department Name
- Files (ILF): Employee, Department
- 2 FTR: Employee, Department
- 3 DET: Name (Employee), Birth Date (Employee), Department Name (Department)

EQ		#DET		
		1-5	6-19	> 19
#FTR	1	low (3)	low (3)	average (4)
	2-3	low (3)	average (4)	high (6)
	> 3	average (4)	high (6)	high (6)

12/01/2026

65

65

Complexity Assessment: ILF

ILF		#DET		
		1-19	20-50	> 50
#RET	1	low (7)	low (7)	average (10)
	2-5	low (7)	average (10)	high (15)
	> 5	average (10)	high (15)	high (15)

- Identify number of Record Types (RET)
- Identify number of Data Element Type (DET)
- Determine complexity weight (→ used to calculate FP count)

12/01/2026

66

66

ILF – Example



ILF		#DET		
		1-19	20-50	> 50
#RET	1	low (7)	low (7)	average (10)
	2-5	low (7)	average (10)	high (15)
	> 5	average (10)	high (15)	high (15)

12/01/2026

67

67

Complexity Assessment: EIF

EIF		#DET		
		1-19	20-50	> 50
#RET	1	low (5)	low (5)	average (7)
	2-5	low (5)	average (7)	high (10)
	> 5	average (7)	high (10)	high (10)

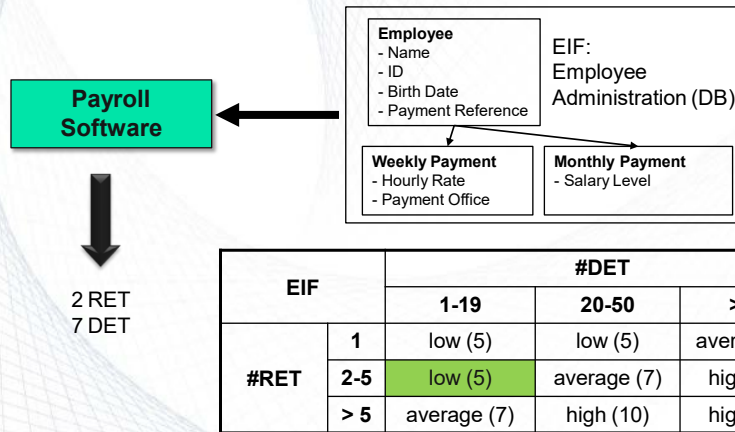
- Identify number of Record Types (RET)
- Identify number of Data Element Type (DET)
- Determine complexity weight (→ used to calculate FP count)

12/01/2026

68

68

EIF – Example

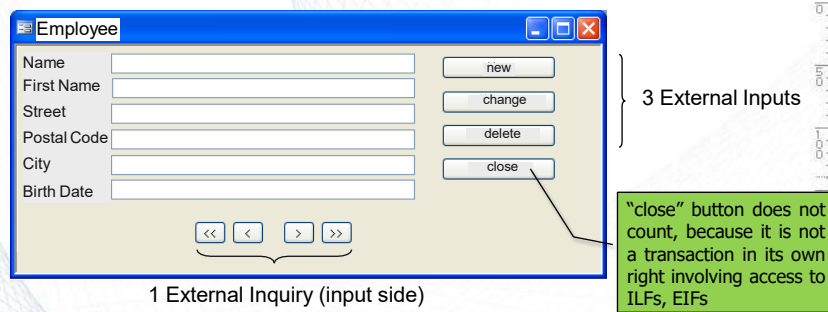


12/01/2026

69

69

FP Counting Example –GUI

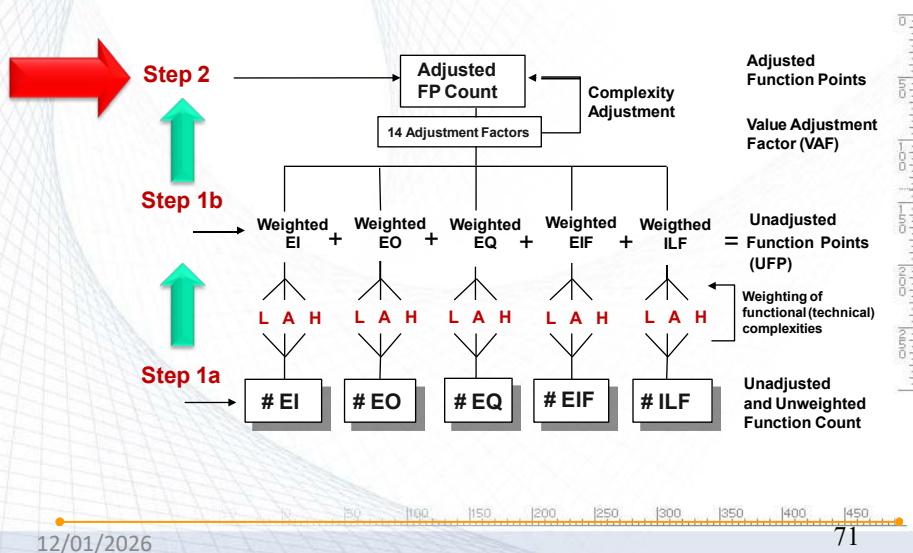


12/01/2026

70

70

FP Counting Summary



71

Value Adjustment Factor (VAF)

/1

- Value Adjustment Factor (*VAF*) (aka. Technical complexity factors) is a weighted sum of 14 components, given below:

F1	Data Communications	F8	On-line Update
F2	Distributed Data Processing	F9	Complex Processing
F3	Performance	F10	Reusability
F4	Heavily Used Configuration	F11	Installation Ease
F5	Transaction Rate	F12	Operational Ease
F6	On-line Data Entry	F13	Multiple Sites
F7	End-User Efficiency	F14	Facilitate Change

12/01/2026

72

72

Value Adjustment Factor (VAF)

/2

- Each component is rated from 0 to 5, where 0 means the component is not relevant to the system and 5 means the component is essential.
- VAF can then be calculated as:

$$VAF = 0.65 + 0.01 \sum_{j=1}^{14} F_j$$

Min = 0
Max = 0.7

- VAF varies from 0.65 (if all F_j are set to 0) to 1.35 (if all F_j are set to 5)

12/01/2026

73

73

Summary

Watch the following video and answer the below question.

- <https://www.youtube.com/watch?v=Nf8XsGjcDSk>

Q. What are FP analysis steps?

12/01/2026

74

74

Software Size: Comparison

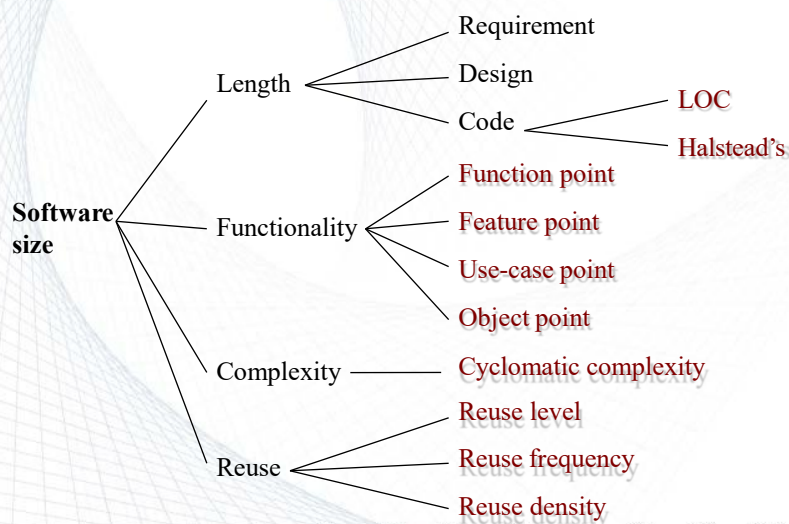
<i>Criteria</i>	<i>LOC</i>	<i>Halstead's</i>	<i>FP</i>	<i>OP</i>
Applicable to early stages (requirement phase) of software development?	<i>No</i>	<i>No (may be)</i>	<i>Yes</i>	<i>Yes</i>
Subjective measure?	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Yes</i>
Derivable from system requirements?	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Yes</i>
Language dependable?	<i>Yes</i>	<i>Yes</i>	<i>No</i>	<i>No</i>
Has clear and understandable physical meaning?	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>No</i>

12/01/2026

75

75

Software Size Metrics: Summary



12/01/2026

76

76

1. Consider a project with the following functional units :

- Number of user inputs = 60
- Number of user outputs = 40
- Number of user enquiries = 30
- Number of user files(DB) = 05
- Number of external interfaces = 04

Assuming all complexity adjustment factors (14 factors have the same value – average) and complexity weighing factors as average, calculate the function points for the project (yêu cầu ghi rõ công thức tính mới được chấm điểm).

(AVERAGE complexity weights = {4, 5, 4, 10, 7} for the 5 complexities respectively.

F1=F2=F6=F8 =2, F5=F3=3, the rest of F is 0.

12/01/2026

77

77

12/01/2026

78

78

References

- Albrecht, A. J., & Gaffney, J. E. (1983). Software function, source lines of code, and development effort prediction: a software science validation. *IEEE transactions on software engineering*, (6), 639-648.
- Function Points Analysis Training Manual (110 Pages)-
<https://www.softwaremetrics.com/freemanual.htm>
- Park, R. E. (1992). *Software size measurement: A framework for counting source statements* (No. CMU/SEI/92-TR-20). Carnegie-Mellon Univ Pittsburgh PA Software Engineering Inst. (available at URL: <http://www.sei.cmu.edu/pub/documents/92.reports/pdf/tr20.92.pdf>)
- <https://www.geeksforgeeks.org/software-engineering-project-size-estimation-techniques/>

12/01/2026

79

79

Bài tập

Suppose that, an add new HR form has 20 input fields (EI), a grid report displays 15 information fields of newly added personnel. Data in this form accessed from 4 tables of the application and referenced to 02 tables of the salary management system. The requirement to check the position salary will be queried from the salary system.

Assume that, the technical complexity of this new HR function is $F1 = F3 = F5 = F7 = 1$, $F4 = F8 = F9 = 2$, the remaining F_s are 0.

Please calculate the functional point (FP) for the add new HR form described above.

12/01/2026

80

80